USED MEDIAS LLC
EMBEDDED SYSTEMS DIVISION

Locking Primitives

UM-CYDOS-014

DOCUMENT	REVISION	STATUS	CLASSIFICATION
UM-CYDOS-014	1.0 · 2026-08-14	**Complete, verified on hardware**	Internal

1. Abstract

cyd-os had no way to make anything mutually exclusive. UM-CYDOS-010 §8 recorded the heap as task-context-only with no locking; UM-CYDOS-013 §7 recorded console output interleaving as the first thing that would need a lock the kernel did not have. Both are now closed.

Two primitives were built, not one, because the two motivating problems have opposite shapes: the heap needs a handful of memory operations to be indivisible, and the console needs a 13-millisecond write to be indivisible. Solving both with the same tool would mean either an unsafe heap or a wrecked scheduler.

The blocking mutex needed three attempts. The second was correct and starved a task to a standstill; §5 is that measurement.

2. Two primitives

	Critical section	Blocking mutex
Mechanism	Raise `PS.INTLEVEL` to mask the tick	Block the task, remove it from the rotation
Cost while held	Scheduling latency, for the whole duration	Two context switches per handoff
Correct for	A few memory operations	Anything longer
Usable from an ISR	Yes	**No** — a handler cannot block
Used by	`heap_alloc`, `heap_free`	Console, application-shared state

The critical section is the wrong tool for the console. Writing a 150-character line at 115200 baud takes about 13 ms. Masking the tick for that long to solve a cosmetic interleaving problem would degrade every scheduling deadline in the system. That asymmetry is why both exist.

`crit_enter()` returns the previous interrupt level and `crit_exit()` restores it, rather than forcing 0. Nesting therefore works, and a section inside an interrupt handler cannot silently re-enable interrupts on the way out.

3. Blocking, and what it required

A mutex that blocks needs the scheduler to understand "not runnable", which it did not. Three changes:

- **`TASK_BLOCKED`**. The scheduler skips it. `TASK_READY` was previously the only live state (UM-CYDOS-010 §8).

- `task_block()` **does not yield.** The caller marks itself blocked while holding a critical section — so the decision to block and the record of what it waits for cannot be split by a tick — then leaves the section and yields. That split is the classic lost wakeup: the holder releases, sees an empty wait list, and the waiter sleeps forever.
- **An idle task.** With blocking, a moment where every task is waiting has nothing to switch to, and the old fallback of resuming the interrupted task would run a task that had just blocked. Idle is registered via `task_set_idle()` and kept **out of the round robin**, chosen only when nothing else can run — leaving it in the rotation would spend a seventh of the machine deliberately doing nothing. It executes `WAITI 0`, which stops the core until an interrupt arrives.

4. Ownership sentinel

`MUTEX_FREE` is `-2`, not `-1`. Before the scheduler starts, `task_current()` returns `-1`, so using `-1` for "unheld" would make an unowned mutex indistinguishable from one held by the boot context, and the free and recursive tests would both match. A one-line problem that would have produced a mutex quietly granting itself to everyone during startup.

The mutex is **recursive**. A console lock is exactly the kind of thing that acquires a nested acquisition by accident, and self-deadlock on a kernel with no debugger is an expensive way to discover it.

5. The starvation defect

The first working mutex released the lock and woke every waiter to race for it. The reasoning recorded in the source was that with at most eight tasks a thundering herd was not worth managing.

That reasoning was wrong, and the instrumentation said so precisely:

```
lock owner=1  waiters=0x00000004  acq=238542  contended=137  err=0  states=1121111
work a/b=238541/0
```

Worker-a had acquired the lock **238,542 times**. Worker-b — `waiters` bit 2, state 2, `TASK_BLOCKED` — had acquired it **zero times**, and had not completed a single iteration of its loop.

This was not a lost wakeup. Worker-b was being woken correctly and re-blocking every time. Worker-a's loop is tight enough that it re-acquired the lock long before the woken waiter was next scheduled. **Race-on-release provides no ordering guarantee at all**, and against a tight loop that is not merely unfair — it starves the waiter indefinitely.

5.1 Fix — direct handoff

`mutex_unlock()` now transfers ownership to a waiter rather than freeing the lock. The successor is the owner *before* it runs again, so nothing can cut in front.

This reintroduces the complication the first design was avoiding: `owner` names a task that is not yet on the CPU. A `granted` bitmask resolves it. A task woken by handoff sees its bit set and returns holding the lock, and that test is made **first** — before the `owner == me` test, which would otherwise read a handoff as a recursive acquisition and leave `depth` permanently too high.

5.2 The cost of fairness

With the handoff in place and the lock taken on *every* worker iteration:

```
work a/b=2062/2061      (was 238541/0)
```

Fair, but throughput collapsed roughly thirtyfold. A handoff costs a full scheduling round-trip — the successor cannot run until the next tick — so a lock contended on every iteration becomes bounded by the tick rate rather than by the work.

That is a real property of a blocking lock, not a defect, but it is a pathological workload. Contending once every 32 iterations is representative, and restores throughput:

```
work a/b=50143/50143    acq=3133  contended=200  err=0  skew=0
```

Both workers now advance **exactly** in step. The lesson worth keeping is that a blocking mutex is the wrong instrument for a lock contended at high frequency; that case wants a critical section, or a design that does not share.

6. Results

```
[6a] critical : PASS  ticks held at 1 across 2 periods, resumed at 7
[6b] mutex    : PASS  recursive depth, ownership, non-owner unlock refused (1),
                      try_lock both ways

t=809  work a/b=50143/50143  guards=ok  corrupt=0
       lock owner=1 waiters=0x00000004 acq=3133 contended=200 err=0 skew=0
       states=1121111
```

6.1 Critical sections mask the tick

Held across two full tick periods with the tick count frozen at 1, then resumed at 7 once the level dropped. Both halves matter: freezing proves interrupts were masked, and resuming proves the tick was **deferred rather than lost**. Masking that silently dropped ticks would keep the scheduler running while making every timeout wrong.

This test originally ran in `m6_selftest()` before `timer_start()`, where `timer_ticks()` is 0 and stays 0 regardless of what masking does — a test that could only ever pass by accident. It reported FAIL, which is the one outcome that made the flaw visible. It now runs from the reporter task with the tick live.

6.2 Mutex semantics

Deterministic, single-threaded: unheld at init, ownership recorded on acquire, recursive acquisition raising depth to 2 without deadlock, depth unwinding correctly, release returning to `MUTEX_FREE`, unlock by a non-owner **refused and counted** rather than acted on, and `try_lock` succeeding when free and failing when held.

Refusing a non-owner unlock matters more than it looks: clearing another task's ownership would admit two holders to the protected section, corrupting whatever it guards far from where the mistake was made.

6.3 Mutual exclusion under contention

`skew=0` throughout. Each worker increments a shared counter under the lock through a deliberately non-atomic read-modify-write with a widened window; the kernel checks that counter against the workers' own independently maintained tallies. Unprotected, this drifts by thousands within a second.

`err=0` — no non-owner unlocks. Task states `1121111` show one task blocked and the rest ready, alternating as the lock changes hands.

6.4 Console arbitration

`ps` output arrived as four unbroken lines with no report interleaved, where previously a report would land mid-table.

Locking is at **message granularity**, not per character. `uart_putc()` stays lock-free, and the panic handler ignores this layer entirely — it runs after a fault, possibly with the lock held by the task that faulted, and a panic that deadlocks trying to report a panic is the worst failure mode available to a kernel with no debugger.

6.5 Regression

M3, M4 and M5 self-tests all still passing. Stack guards intact, `corrupt=0`, headroom 449 words.

7. Metrics

Quantity	Value
Primitives	2
Critical section cost	`RSR` / `WSR` on `PS`, no memory
Mutex size	24 B
Tasks	7 (6 working + idle)
Lock acquisitions measured	3,133
Contentions	200
Non-owner unlocks	0
Lost updates	0
Throughput, pathological contention	~2,060 iterations
Throughput, 1-in-32 contention	~50,143 iterations
Image size	16,256 B

8. What this does not establish

- **No priority inheritance.** There are no priorities, so priority inversion cannot occur — but it will the moment priorities are added, and this mutex has no answer for it.
- **No deadlock detection.** Two tasks taking two mutexes in opposite orders will hang, and the kernel will not say so. The idle task will simply run.

- **No timeouts.** `mutex_lock()` waits forever.
- **No condition variables or semaphores.** There is no way to wait for an event, only for a lock.
- **Keystroke echo is not arbitrated.** The shell echoes characters outside the lock, so a report can still split a partially typed line. Locking each keystroke would serialise the console against a human typing speed.
- **Critical sections are unbounded by convention only.** Nothing measures or enforces how long one is held; a long one silently degrades scheduling.
- **Untested against an ISR.** The mutex is documented as unusable from an interrupt handler and nothing enforces that.

9. References

- UM-CYDOS-010 §8 — the heap's task-context-only limitation, closed here
- UM-CYDOS-013 §7 — console interleaving, closed here
- UM-CYDOS-009 — the scheduler that `TASK_BLOCKED` extends
- `kernel/critical.h` — masking, and why it is wrong for long sections
- `kernel/mutex.c` — handoff, the `granted` bitmask, and §5
- `kernel/console.c` — message-granularity arbitration